

Master Thesis Project

On writing high-assurance code against timing-attacks in the presence of speculative execution

Contact: Ahmed Rezine¹ and Niklas Johansson²

¹ Linköping University

² Sectra

Side channel attacks represent an important security threat for many systems. Typical side-channel attacks aim to recover cryptography keys and private information. The idea is to use non-functional characteristics of the computations in order to recover secret information. Such characteristics can include the time computations may take, their cache behavior, their power consumption or their memory usage [1]. Of particular interest for this work are cache-based timing attacks in the presence of speculative execution.

Such attacks rely on the fact that modern architectures speculate (e.g., on the outcome of if-statements) to avoid waiting. However, by the time the execution is aborted, micro-architectural aspects of the execution (here caches) might have been updated based on secret information. This cache state can be assessed by an attacker to deduce secret information (e.g., cryptography keys).

Recent research produced different well founded approaches to reason about these issues. On the one hand, tools such as Spectector [2] would start from (assembly) code (e.g., of cryptography libraries) and check whether it can leak more secret information during symbolic executions. On the other hand, frameworks such as the Jasmin framework [3] allow the user to write the program in a domain specific language where proofs can be conducted and assembly code can be generated.

This project will experiment with at least one representative of each approach and assess the advantages and disadvantages of each approach.

For this purpose, the project aims to:

1. Gain familiarity with cache-based side-channel vulnerabilities under speculation.
2. Gain familiarity with verification based approaches for uncovering them.
3. Collect relevant source code and assess the applicability of such formal verification based approaches.

Qualification

This 30 hp thesis will be carried by one or two master student(s).

- The student should have very good programming skills in C
- The student should have taken a compilers and a computer hardware course
- Having taken the software verification course and experience with functional programming are a plus

References

1. Sudipta Chattopadhyay, Moritz Beck, Ahmed Rezzine, and Andreas Zeller. Quantifying the information leakage in cache attacks via symbolic execution. *ACM Transactions on Embedded Computing Systems (TECS)*, 18(1):1–27, 2019.
2. Marco Guarnieri, Boris Köpf, José F Morales, Jan Reineke, and Andrés Sánchez. Spectector: Principled detection of speculative information flows. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1–19. IEEE, 2020.
3. Basavesh Ammanaghatta Shivakumar, Gilles Barthe, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Swarn Priya, Peter Schwabe, and Lucas Tabary-Maujean. Typing high-speed cryptography against spectre v1. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 1094–1111. IEEE, 2023.